

RETREIVE – AI-Powered Active Reading Platform

Project Overview

Retreive is a web application that helps students improve retention by reading study material aloud.

Users upload educational PDFs (MCAT initially, but the platform should be generic enough for future expansion). Instead of reading raw PDF content, the system extracts the content, identifies complete topics or chapters, and uses an AI model to rewrite the material into a cleaner and more readable format.

The rewritten content becomes the reading material.

As users read aloud, the application tracks spoken words in real time, highlights text karaoke-style, saves progress continuously, and tests comprehension using MCQs.

The primary goal is to improve retention through active reading and immediate recall.

Core Principles

1. Users never read raw PDF text.
 2. AI-generated content is the reading source.
 3. Progress must never be lost.
 4. Every completed paragraph must be saved immediately.
 5. Content should be split by complete topics, not arbitrary page counts.
 6. The experience must be mobile-friendly and responsive.
 7. The application is completely free in V1.
 8. No payment system is required.
-

Technology Stack

Frontend

- React
- React Router
- Axios
- Tailwind CSS

Backend

- Node.js

- Express.js

Database

- MongoDB Atlas

File Storage

- Cloudinary

Authentication

- JWT
- bcrypt

Speech Recognition

- Deepgram

AI Processing

One of:

- Gemini
- Claude
- OpenAI

AI provider should be abstracted behind a service layer so models can be switched later.

Deployment

Frontend:

- Vercel

Backend:

- Render or Railway

Database:

- MongoDB Atlas
-

High-Level User Flow

User signs up



User uploads PDF



PDF stored in Cloudinary



Text extracted



Topic/chapter boundaries detected



System selects complete study section



AI rewrites content



Structured reading passages generated



Reading session begins



Deepgram tracks speech



Words highlight in real time

↓

Progress saves continuously

↓

MCQs appear after passages

↓

Analytics generated

Authentication Module

Users can:

- Sign up
- Log in
- Log out
- Access protected routes

Required fields:

```
{
  "name": "",
  "email": "",
  "password": ""
}
```

JWT authentication should be used.

PDF Processing Module

Upload

User uploads PDF.

Backend stores PDF in Cloudinary.

Store metadata in MongoDB.

Extraction

Extract text from PDF.

Use:

- pdf-parse
- equivalent extraction service

Validation

Reject PDFs where meaningful text cannot be extracted.

Example:

Scanned image-only PDFs should return a helpful error.

Topic Detection Module

The system must identify complete sections.

Examples:

Good:

- Chapter 3: Cell Biology
- The Immune System
- Protein Synthesis

Bad:

- Half of one chapter
- Random 10 pages
- Content cut mid-topic

The goal is to create complete learning units.

Possible techniques:

- Heading detection
- Font size analysis
- LLM topic segmentation
- Semantic chunking

The implementation can evolve.

AI Rewriting Module

After extraction and topic selection:

Send content to AI.

AI should:

- Simplify language
- Improve readability
- Preserve scientific accuracy
- Keep educational intent
- Remove formatting noise
- Create smooth reading passages

AI should NOT:

- Hallucinate new facts
- Change scientific meaning
- Omit critical concepts

Output should be structured paragraphs.

Store both:

- Original extracted text
- AI rewritten text

for debugging and future comparison.

Reading Engine

This is the core feature.

Users read rewritten content aloud.

Requirements:

- Real-time word tracking
- Karaoke-style highlighting
- Large readable text
- Dark theme

Speech recognition provider:

Deepgram

Reading progress should only advance while speech is detected.

Session Persistence

Critical requirement.

Every paragraph completion must immediately save progress.

Never wait for:

- session end
- browser close
- page refresh

Save immediately.

Users must resume exactly where they left off.

Stored state includes:

- Current passage
 - Current paragraph
 - Current word index
 - Progress percentage
 - Words read
 - Time spent
-

MCQ Module

After passage completion:

Show 2 MCQs.

Purpose:

Retention testing.

For each question:

Correct answer:

- Success state
- Positive reinforcement

Wrong answer:

- Show correct answer
- Explain reasoning

Store all attempts.

Track accuracy.

Analytics Module

Provide basic analytics.

Metrics:

- Total words read
- Reading time
- Accuracy percentage
- Passages completed
- Sessions completed

Do not build advanced dashboards in V1.

Simple summary screens only.

Science Facts Module

Show educational retention facts.

Displayed before reading sessions.

Requirements:

- Randomized order
- No repeats until cycle completes
- Dark theme
- Title
- Description

- Emoji

User can skip.

Database Collections

users

```
{
  _id,
  name,
  email,
  passwordHash,
  createdAt,
  updatedAt
}
```

pdfs

```
{
  _id,
  userId,
  originalName,
  cloudinaryUrl,
  cloudinaryPublicId,
  totalPages,
  uploadDate
}
```

processedContents

```
{
  _id,
  userId,
  pdfId,

  sourceStartPage,
  sourceEndPage,

  detectedChapter,
  detectedTopic,
}
```

```
    rawText,  
    aiText,  
  
    passages: [],  
  
    aiModel,  
  
    status,  
  
    createdAt  
}
```

sessions

```
{  
  _id,  
  
  userId,  
  pdfId,  
  processedContentId,  
  
  currentPassage,  
  currentParagraph,  
  currentWord,  
  
  wordsRead,  
  timeSpent,  
  
  accuracy,  
  
  progressPercent,  
  
  status,  
  
  startedAt,  
  updatedAt  
}
```

mcqs

```
{  
  _id,  
  processedContentId,
```

```
    passageIndex,  
  
    question,  
    options,  
  
    correctAnswer,  
  
    explanation  
}
```

mcqAttempts

```
{  
  _id,  
  
  userId,  
  sessionId,  
  mcqId,  
  
  selectedAnswer,  
  
  isCorrect,  
  
  createdAt  
}
```

scienceCards

```
{  
  _id,  
  title,  
  body,  
  emoji  
}
```

Development Phases

Phase 1

Authentication Module

Deliverables:

- Signup
 - Login
 - JWT auth
 - Protected routes
-

Phase 2

PDF Upload Module

Deliverables:

- Upload PDF
 - Store in Cloudinary
 - Save metadata
-

Phase 3

PDF Extraction Module

Deliverables:

- Extract text
 - Validate extraction
 - Save raw content
-

Phase 4

Topic Detection Module

Deliverables:

- Detect chapters
 - Detect topic boundaries
 - Generate complete study sections
-

Phase 5

AI Rewriting Module

Deliverables:

- Rewrite extracted content
 - Generate structured passages
 - Store processed content
-

Phase 6

Reading Engine

Deliverables:

- Passage reader
 - Deepgram integration
 - Word highlighting
 - Reading progress
-

Phase 7

Session Persistence

Deliverables:

- Immediate paragraph saves
 - Resume functionality
 - Progress restoration
-

Phase 8

MCQ System

Deliverables:

- Generate or store MCQs
 - Answer evaluation
 - Accuracy tracking
-

Phase 9

Analytics

Deliverables:

- Reading statistics
 - Session summary
-

Phase 10

Science Cards + UI Polish

Deliverables:

- Science card rotation
 - Responsive UI
 - Mobile optimization
-

Phase 11

Testing & Deployment

Deliverables:

- End-to-end testing
 - Production deployment
 - Bug fixes
 - Performance optimization
-

Important Notes For Any AI Coding Agent

1. Prioritize modular architecture.
2. Use service layers for AI integrations.
3. Keep AI provider interchangeable.
4. Optimize MongoDB schema for large PDFs.
5. Build APIs before UI where possible.
6. Session persistence is a critical feature.
7. Never expose API keys.
8. Focus on maintainability over shortcuts.
9. Use TypeScript if possible.
10. Treat this document as the source of truth unless explicitly overridden.